AMBIENTE VIRTUAL CLIENTE/SERVIDOR COM DOCKER: IMPLANTAÇÃO DE UMA APLICAÇÃO WEB DE TAREFAS

Elias Eder Gasparassi[1]

RESUMO

Este trabalho relata a implantação de um ambiente virtual cliente/servidor para o desenvolvimento de aplicações web, construído inteiramente com softwares livres. Utilizou-se a virtualização em nível de sistema operacional por meio de contêineres Docker, executados sobre o Linux Mint, e o ambiente foi descrito de forma declarativa com o Docker Compose. A arquitetura é composta por quatro serviços isolados e interligados por uma rede virtual: o servidor web Nginx, atuando como proxy reverso; o interpretador PHP-FPM, que hospeda uma API REST; o sistema gerenciador de banco de dados MySQL; e um servidor de arquivos estáticos para a interface do usuário. A metodologia consistiu na instalação das ferramentas, na escrita dos arquivos de configuração de cada serviço, na construção das imagens e na validação do funcionamento por meio de uma aplicação de gerenciamento de tarefas (ToDo), com interface desenvolvida em HTML, JavaScript e Bootstrap. Os resultados mostram que o ambiente pode ser reproduzido com um único comando, consome poucos recursos e reproduz a separação de responsabilidades encontrada em ambientes de produção.

Palavras-chave: Docker. Nginx. Software livre.

1 INTRODUÇÃO

O desenvolvimento de aplicações web exige um ambiente composto por diversos serviços: um servidor web que receba as requisições HTTP, um interpretador para a linguagem do lado do servidor e um sistema gerenciador de banco de dados. Instalar todos esses componentes diretamente na máquina do desenvolvedor costuma gerar conflitos de versões, dificuldade de reprodução e diferenças entre o ambiente de desenvolvimento e o de produção.

A virtualização surge como resposta a esse problema, permitindo isolar cada serviço em um ambiente próprio. Entre as abordagens disponíveis, os contêineres destacam-se por compartilharem o núcleo do sistema operacional hospedeiro, o que os torna mais leves e rápidos que as máquinas virtuais tradicionais (MERKEL, 2014).

O objetivo deste trabalho é implantar um ambiente virtual cliente/servidor de desenvolvimento para aplicações web utilizando somente softwares livres e validá-lo com uma aplicação real de gerenciamento de tarefas. Como objetivos específicos, buscou-se: descrever o ambiente de forma declarativa e reproduzível; separar os

[1] Aluno(a) da Turma 2026 da Pós-Graduação em Desenvolvimento Web do IFSULDEMINAS.

serviços de servidor web, aplicação e banco de dados em contêineres distintos; e documentar o processo de construção.

2 SOFTWARES E FERRAMENTAS UTILIZADAS

Esta seção apresenta o ambiente de virtualização, o sistema operacional e os softwares livres empregados na implantação.

2.1 Ambiente de virtualização

Foi adotado o **Docker**, plataforma de código aberto que empacota aplicações e suas dependências em imagens e as executa como contêineres isolados (DOCKER, 2026). Diferentemente de um *hipervisor*, que emula um computador completo com seu próprio sistema operacional, o Docker utiliza recursos do núcleo Linux, como *namespaces* e *cgroups*, para isolar processos, o que reduz o consumo de memória e o tempo de inicialização (MERKEL, 2014).

Para orquestrar os múltiplos contêineres foi utilizado o **Docker Compose**, que permite descrever, em um único arquivo YAML, todos os serviços, redes e volumes do ambiente, criando-os e iniciando-os com um único comando.

2.2 Sistema operacional

O sistema hospedeiro foi o **Linux Mint 22.3**, distribuição livre baseada no Ubuntu e no Debian (LINUX MINT, 2026). Dentro dos contêineres foram utilizadas imagens baseadas no **Alpine Linux**, distribuição mínima voltada à segurança e ao baixo consumo de recursos, o que resulta em imagens de poucos *megabytes*.

2.3 Softwares livres utilizados

O Quadro 1 resume os softwares livres utilizados, suas versões e a função de cada um no ambiente.

Software	**Versão / Imagem**	**Função no ambiente**
Docker Engine	29.9.3	Execução dos contêineres
Docker Compose	v5.7.0	Orquestração dos serviços
Nginx	nginx:1.31-alpine	Servidor web e proxy reverso
PHP-FPM	php:8.5-fpm-alpine	Execução da API em PHP
MySQL Community	mysql:8.1	Banco de dados relacional
Bootstrap	5.3.3	Componentes da interface web

Quadro 1 – Softwares livres utilizados no ambiente
Fonte: Elaborado pelo autor (2026).

O **Nginx** é um servidor HTTP de alto desempenho, capaz de atuar também como proxy reverso e balanceador de carga (NGINX, 2026). O **PHP-FPM** (FastCGI Process Manager) é a implementação do PHP que atende requisições pelo

protocolo FastCGI, mantendo um conjunto de processos prontos para executar os scripts (PHP GROUP, 2026). O **MySQL** é um dos sistemas gerenciadores de banco de dados relacionais mais usados em aplicações web (ORACLE, 2026). Por fim, o **Bootstrap** é um framework livre de componentes HTML, CSS e JavaScript para interfaces responsivas (BOOTSTRAP, 2026).

3 CONSTRUÇÃO DO AMBIENTE VIRTUAL

A construção iniciou-se com a instalação do Docker no Linux Mint a partir dos repositórios da distribuição, com os pacotes docker.io e docker-compose-v2. Em seguida, o usuário foi incluído no grupo docker, permitindo executar os comandos sem privilégios de administrador, e a instalação foi conferida com os comandos *docker --version* e *docker compose version*.

O projeto foi organizado em um diretório por serviço (nginx, php, mysql e frontend), cada um com seus arquivos de configuração e, quando necessário, um Dockerfile próprio, que descreve como a imagem do serviço é construída. Na raiz, o arquivo docker-compose.yml define os quatro serviços, a rede virtual do tipo bridge chamada todo-network, que permite a comunicação entre os contêineres pelo nome do serviço, e um volume persistente para os dados do MySQL. A Figura 1 apresenta a arquitetura resultante.

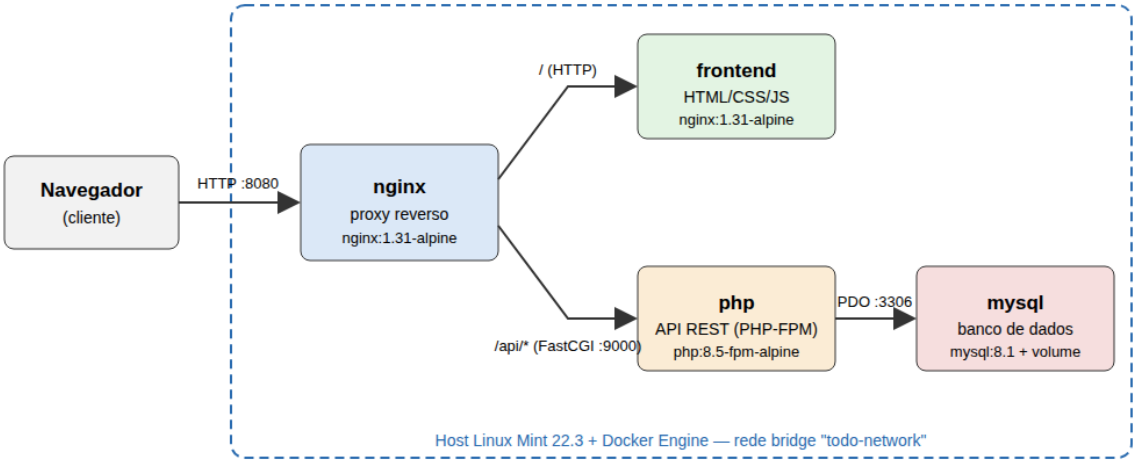


Figura 1 – Arquitetura do ambiente virtual
Fonte: Elaborado pelo autor (2026).

As principais configurações de cada serviço foram:

- **nginx**: é o único contêiner exposto ao hospedeiro, na porta 8080. Seu arquivo nginx.conf encaminha as requisições da raiz (/) ao contêiner frontend via HTTP e as rotas /api/* ao contêiner php pelo protocolo FastCGI, na porta 9000;

- **php**: imagem oficial do PHP-FPM acrescida das extensões pdo e pdo_mysql. O código-fonte é montado no contêiner por *bind mount*, de modo que alterações no código têm efeito imediato, sem reconstruir a imagem;
- **mysql**: imagem oficial do MySQL, configurada por variáveis de ambiente (banco, usuário e senha). O script init.sql cria a tabela de tarefas na primeira inicialização, e uma verificação de saúde (*healthcheck*) garante que o PHP só seja iniciado quando o banco estiver pronto;
- **frontend**: servidor Nginx que entrega os arquivos estáticos da interface (HTML, CSS e JavaScript).

Com os arquivos prontos, o ambiente foi construído e iniciado com o comando *docker compose up -d --build*, que constrói as imagens, cria a rede e o volume e inicia os contêineres em segundo plano. O estado dos serviços foi conferido com *docker compose ps*, e o funcionamento da API foi testado pelo terminal com o utilitário *curl*, antes mesmo do acesso pelo navegador. O comando *docker compose down* encerra o ambiente sem perder os dados, que permanecem no volume.

4 APLICAÇÃO WEB

Para validar o ambiente foi desenvolvida uma aplicação de gerenciamento de tarefas (ToDo), na qual o usuário pode cadastrar, listar, marcar como concluídas e excluir tarefas. A aplicação segue o modelo cliente/servidor: a interface, executada no navegador, consome uma API REST escrita em PHP, que persiste os dados no MySQL.

A API foi implementada em um único script PHP, que identifica a rota e o método HTTP da requisição e responde em formato JSON. O acesso ao banco é feito com a extensão PDO e consultas preparadas, o que evita ataques de injeção de SQL. As credenciais de conexão são lidas de variáveis de ambiente definidas no *docker-compose.yml*. O Quadro 2 apresenta as rotas disponíveis.

Método	**Rota**	**Ação**
GET	/api/tasks	Lista todas as tarefas
POST	/api/tasks	Cria uma tarefa a partir do título informado
PUT	/api/tasks/{id}	Alterna a tarefa entre pendente e concluída
DELETE	/api/tasks/{id}	Exclui a tarefa

Quadro 2 – Rotas da API REST da aplicação
Fonte: Elaborado pelo autor (2026).

A interface foi construída com HTML, JavaScript puro e o *framework* Bootstrap. Ela oferece barra de progresso, filtros por situação com contadores de

tarefas, confirmação antes da exclusão, notificações de sucesso e erro e alternância entre tema claro e escuro. Como o Nginx atende tanto a interface quanto a API pelo mesmo endereço, não são necessárias configurações de compartilhamento entre origens (CORS). A Figura 2 mostra a aplicação em execução no ambiente.

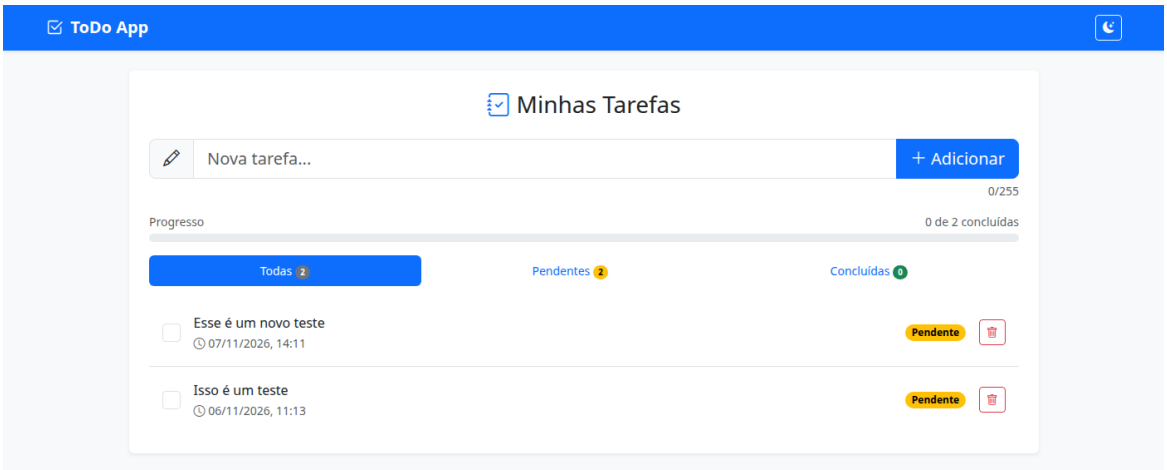


Figura 2 – Aplicação ToDo em execução no ambiente virtual
Fonte: Elaborado pelo autor (2026).

5 CONCLUSÃO

O trabalho atingiu o objetivo de implantar um ambiente virtual cliente/servidor para o desenvolvimento de aplicações web utilizando somente softwares livres. O uso do Docker e do Docker Compose permitiu descrever toda a infraestrutura em arquivos de texto, versionáveis e reproduzíveis em qualquer máquina com Docker instalado, com um único comando.

A separação dos serviços em contêineres distintos, comunicando-se por uma rede virtual, reproduziu a arquitetura típica de ambientes de produção, com o Nginx como ponto único de entrada. A aplicação de tarefas comprovou o funcionamento integrado de todos os componentes.

Como limitações, destacam-se as credenciais definidas diretamente no arquivo de orquestração e a ausência de HTTPS, aceitáveis apenas em ambiente acadêmico. Como trabalhos futuros, sugerem-se o uso de certificados TLS no Nginx, o gerenciamento seguro de segredos e a implantação do ambiente em um servidor na nuvem com integração contínua.

6 REFERÊNCIAS

BOOTSTRAP. **Bootstrap documentation**. Disponível em: https://getbootstrap.com/docs/5.3/. Acesso em: 07 nov. 2026.

DOCKER. **Docker documentation**. Disponível em: https://docs.docker.com/. Acesso em: 07 nov. 2026.

LINUX MINT. **Linux Mint user guide**. Disponível em: https://linuxmint.com/documentation.php. Acesso em: 07 nov. 2026.

MERKEL, Dirk. Docker: lightweight Linux containers for consistent development and deployment. **Linux Journal**, Houston, v. 2014, n. 239, mar. 2014.

NGINX. **nginx documentation**. Disponível em: https://nginx.org/en/docs/. Acesso em: 07 nov. 2026.

ORACLE. **MySQL 8.0 reference manual**. Disponível em: https://dev.mysql.com/doc/refman/8.0/en/. Acesso em: 07 nov. 2026.

PHP GROUP. **PHP manual: FastCGI Process Manager (FPM)**. Disponível em: https://www.php.net/manual/pt_BR/install.fpm.php. Acesso em: 07 nov. 2026.